

Karpenter upgrade to v1beta1 version

To upgrade karpenter to v1beta1 , will need to upgrade our existing alpha karpenter to v0.31.4 (Please note **v0.31.4 is the only patch version that supports rollback for v1beta1.**)

Check your current karpenter version:

```
1 kubectl get pod -A | grep karpenter
2 kubectl describe pod -n karpenter karpenter-xxxxxxxx-xxxxx | grep Image: | grep v0
```

Sample output:

```
1 image: public.ecr.aws/karpenter/controller:v0.27.3@sha256:87ea68abc00b806e45df8a519eacf845f5f0fa577596edc3be83fad
```

Upgrade to Karpenter v0.31.4 .

First fetch your helm values.yaml file from existing karpenter .

```
opstree@opstree-Latitude-3420:~$ helm ls -n karpenter
NAME          NAMESPACE    REVISION    UPDATED              STATUS    CHART          APP VERSION
karpenter     karpenter     9           2024-02-20 13:10:46.449244 +0530 IST    deployed karpenter-v0.27.3    0.27.3
karpenter-crd karpenter     1           2024-02-20 13:01:01.092178 +0530 IST    deployed karpenter-crd-v0.27.3 0.27.3
opstree@opstree-Latitude-3420:~$ helm -n karpenter get values karpenter
USER-SUPPLIED VALUES:
controller:
  env:
    - name: AWS_ENI_LIMITED_POD_DENSITY
      value: "false"
nodeSelector:
  kubernetes.io/os: linux
serviceAccount:
  create: false
  name: karpenter
serviceMonitor:
  enabled: true
settings:
  aws:
    clusterEndpoint: https://0EF73D94742730D904CE9176BD085860B.sk1.ap-south-1.eks.amazonaws.com
    clusterName: development-eks
    defaultInstanceProfile: null
```

Set environment variables for your cluster:

```
1 export TAG=v0.31.4
2 export CLUSTER_NAME="<cluster-name>"
3 export AWS_ACCOUNT_ID="$(aws sts get-caller-identity --query Account --output text)"
4 export KARPENTER_IAM_ROLE_ARN="<Role:arn>" #existing karpenter role arn
5 export CLUSTER_ENDPOINT="$(aws eks describe-cluster --name ${CLUSTER_NAME} --region ap-south-1 --query "cluster.e
```

Helm command to upgrade to v0.31.4 .

Note : Please add the karpenter value.yaml file in below command

```
1 helm upgrade --install karpenter oci://public.ecr.aws/karpenter/karpenter --version ${TAG} --namespace karpenter
2 --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=${KARPENTER_IAM_ROLE_ARN} \
3 --set settings.aws.clusterName=${CLUSTER_NAME} \
4 --set settings.aws.clusterEndpoint=${CLUSTER_ENDPOINT} \
5 --set settings.aws.defaultInstanceProfile=KarpenterNodeInstanceProfile-${CLUSTER_NAME} \
6 --set settings.aws.interruptionQueueName=${CLUSTER_NAME} \
7 --wait \
8 --atomic \
9 --timeout 2m15s
```

After execution of above helm command check karpenter pod status and image used.

```
1 kubectl get pod -A | grep karpenter
2 kubectl describe pod -n karpenter karpenter-xxxxxxxx-xxxxx | grep Image: | grep v0
```

Note : Please check image version is v0.31.4.

```
1 image: public.ecr.aws/karpenter/controller:v0.31.4@sha256:87ea68abc00b806e45df8a519eacf845f5f0fa577596edc3be83fad
```

Before upgrading to karpenterv1beta1 version, make sure karpenter v0.31.4 is working properly.

Use below yaml file to increase dummy load at cluster to verify working of karpenter.

```
1 ---
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: nginx-deployment
6   labels:
7     app: nginx
8   replicas: 5
9   selector:
10    matchLabels:
11      app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18       - name: nginx
19         image: nginx:1.14.2
20         resources:
21           requests:
22             cpu: "4"
23             memory: 4Gi
24         ports:
25         - containerPort: 80
```

Karpenter v1beta1 migration.

Set environment variables for your cluster:

Note : Please make sure `KARPENTER_IAM_ROLE_ARN` **is already existed before you set the above environment variable, if role is not present please create it manually..**

```
1 export KARPENTER_NAMESPACE=karpenter
2 export KARPENTER_VERSION=v0.32.9
3 export AWS_PARTITION="aws" # if you are not using standard partitions, you may need to configure to aws-cn / aws-
4 export CLUSTER_NAME="<cluster-name>"
5 export AWS_REGION="<region>"
6 export AWS_ACCOUNT_ID="$(aws sts get-caller-identity --query Account --output text)"
7 export KARPENTER_IAM_ROLE_ARN="arn:${AWS_PARTITION}:iam::${AWS_ACCOUNT_ID}:role/${CLUSTER_NAME}-karpenter"
8 export CLUSTER_ENDPOINT="$(aws eks describe-cluster --name ${CLUSTER_NAME} --query "cluster.endpoint" --output te
```

Update `KARPENTER_IAM_ROLE_ARN` trust relationship with below content.

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Federated": "<oidc-identity-provider-arn>"
8       },
9       "Action": "sts:AssumeRoleWithWebIdentity",
10      "Condition": {
11        "StringEquals": {
12          "<provider>:sub": "system:serviceaccount:<karpenter-namespace>:karpenter",
13          "<provider>:aud": "sts.amazonaws.com"
14        }
15      }
16    ]
17  }
18 }
```

Apply the new Karpenter policy and assign it to the existing Karpenter role:

```

1 TEMPOUT=$(mktemp)
2 curl -fsSL https://raw.githubusercontent.com/aws/karpenter-provider-aws/v0.32.9/website/content/en/preview/upgra
3
4 AWS_REGION=${AWS_REGION:=AWS_DEFAULT_REGION} # use the default region if AWS_REGION isn't defined
5 POLICY_DOCUMENT=$(envsubst < ${TEMPOUT})
6 POLICY_NAME="KarpenterControllerPolicy-${CLUSTER_NAME}-v1beta1"
7 ROLE_NAME="${CLUSTER_NAME}-karpenter"
8
9 POLICY_ARN=$(aws iam create-policy --policy-name "${POLICY_NAME}" --policy-document "${POLICY_DOCUMENT}" | jq -r
10 aws iam attach-role-policy --role-name "${ROLE_NAME}" --policy-arn "${POLICY_ARN}")
```

Need to update policy attached with `KARPENTER_IAM_ROLE` as `ec2:terminate` permission is not added in above policy . Please add below block in policy.

```

1 {
2   "Sid": "ec2terminateaccess",
3   "Effect": "Allow",
4   "Resource": "*",
5   "Action": "ec2:TerminateInstances"
6 }
```

Apply the v0.32.9 Custom Resource Definitions (CRDs):

```

1 kubectl apply -f https://raw.githubusercontent.com/aws/karpenter-provider-aws/v0.32.9/pkg/apis/crds/karpenter.sh
2 kubectl apply -f https://raw.githubusercontent.com/aws/karpenter-provider-aws/v0.32.9/pkg/apis/crds/karpenter.sh
3 kubectl apply -f https://raw.githubusercontent.com/aws/karpenter-provider-aws/v0.32.9/pkg/apis/crds/karpenter.k8
4 kubectl apply -f https://raw.githubusercontent.com/aws/karpenter-provider-aws/v0.32.9/pkg/apis/crds/karpenter.sh
```

```
5 kubectl apply -f https://raw.githubusercontent.com/aws/karpenter-provider-aws/v0.32.9/pkg/apis/crds/karpenter.sh
6 kubectl apply -f https://raw.githubusercontent.com/aws/karpenter-provider-aws/v0.32.9/pkg/apis/crds/karpenter.k8
```

Upgrade Karpenter to the new version:

Note : Please add the karpenter value.yaml file in below command

```
1 helm registry logout public.ecr.aws
2
3 helm upgrade --install karpenter oci://public.ecr.aws/karpenter/karpenter --version ${KARPENTER_VERSION} --names
4 --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=${KARPENTER_IAM_ROLE_ARN} \
5 --set settings.aws.defaultInstanceProfile=KarpenterNodeInstanceProfile-${CLUSTER_NAME} \
6 --set settings.clusterName=${CLUSTER_NAME} \
7 --set settings.interruptionQueue=${CLUSTER_NAME} \
8 --set controller.resources.requests.cpu=1 \
9 --set controller.resources.requests.memory=1Gi \
10 --set controller.resources.limits.cpu=1 \
11 --set controller.resources.limits.memory=1Gi \
12 --wait \
13 --atomic \
14 --timeout 2m15s
15
```

Create KARPENTER_NODE_ROLE

```
1 echo '{
2     "Version": "2012-10-17",
3     "Statement": [
4         {
5             "Effect": "Allow",
6             "Principal": {
7                 "Service": "ec2.amazonaws.com"
8             },
9             "Action": "sts:AssumeRole"
10        }
11    ]
12 }' > node-trust-policy.json
13
14 aws iam create-role --role-name "KarpenterNodeRole-${CLUSTER_NAME}" \
15 --assume-role-policy-document file://node-trust-policy.json
16
```

Now attach the required policies to the role.

```
1 aws iam attach-role-policy --role-name "KarpenterNodeRole-${CLUSTER_NAME}" \
2 --policy-arn "arn:${AWS_PARTITION}:iam::aws:policy/AmazonEKSWorkerNodePolicy"
3
4 aws iam attach-role-policy --role-name "KarpenterNodeRole-${CLUSTER_NAME}" \
5 --policy-arn "arn:${AWS_PARTITION}:iam::aws:policy/AmazonEKS_CNI_Policy"
6
7 aws iam attach-role-policy --role-name "KarpenterNodeRole-${CLUSTER_NAME}" \
8 --policy-arn "arn:${AWS_PARTITION}:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
9
10 aws iam attach-role-policy --role-name "KarpenterNodeRole-${CLUSTER_NAME}" \
11 --policy-arn "arn:${AWS_PARTITION}:iam::aws:policy/AmazonSSMManagedInstanceCore"
```

Install the `karpenter-convert` tool to help convert the alpha Karpenter manifests to beta manifests

```
1 go install github.com/aws/karpenter/tools/karpenter-convert/cmd/karpenter-convert@release-v0.32.x
```

Convert each `AWSNodeTemplate` to an `EC2NodeClass`. To convert your `v1alpha` Karpenter manifests to `v1beta1`, you can either manually apply changes to API components or use the `karpenter-convert` CLI tool.

```
1 export KARPENTER_NODE_ROLE="KarpenterNodeRole-${CLUSTER_NAME}"
2 karpenter-convert -f awsnode-template.yaml | envsubst > ec2nodeclass.yaml
```

When you are satisfied with your `EC2NodeClass` file, apply it as follows:

```
1 kubectl apply -f ec2nodeclass.yaml
```

Convert each `Provisioner` to a `NodePool`. Again, either manually update your `Provisioner` manifests or use the `karpenter-convert` CLI tool:

```
1 karpenter-convert -f provisioner.yaml > nodepool.yaml
```

When you are satisfied with your `NodePool` file, apply it as follows:

```
1 kubectl apply -f nodepool.yaml
```

Roll over nodes: With the new `NodePool` yaml in hand, there are several ways you can begin to roll over your nodes to use the new `NodePool`:

1. Periodic Rolling with [Expiration](#):

- Add the following taint to the old `Provisioner`: `karpenter.sh/legacy=true:NoSchedule`
- Set the `Expiration` in your `Provisioner` to a small value like `ttlSecondsUntilExpired: 300` to mark all nodes older than 5 minutes as expired. *Please note that in beta APIs, this is the same as `disruption.expireAfter`*
- Watch as replacement nodes are launched from the new `NodePool` resource.

Because Karpenter will only roll one node at a time, it may take some time for Karpenter to completely roll all nodes under a `Provisioner`. For eg:

```
1 apiVersion: karpenter.sh/v1alpha5
2 kind: Provisioner
3 metadata:
4   annotations:
5     karpenter.sh/provisioner-hash: "3323802356595469889"
6     kubernetes.io/last-applied-configuration: |
7       {"apiVersion":"karpenter.sh/v1alpha5","kind":"Provisioner","metadata":{"annotations":{},"name":"microservice-nodes"},"spec":{"consolidation":{"enabled":false},"kubeletConfiguration":{"containerRuntime":"containerd","maxPods":110}}}
8   generation: 2
9   name: microservice-nodes
10 spec:
11   consolidation:
12     enabled: false
13   kubeletConfiguration:
14     containerRuntime: containerd
15     maxPods: 110
```

```

16     systemReserved:
17         cpu: 350m
18         ephemeral-storage: 4Gi
19         memory: 500Mi
20     providerRef:
21         name: microservice-provider
22     requirements:
23     - key: kubernetes.io/arch
24       operator: In
25       values:
26       - amd64
27     - key: karpenter.sh/capacity-type
28       operator: In
29       values:
30       - spot
31     - key: karpenter.k8s.aws/instance-family
32       operator: In
33       values:
34       - c6i
35       - c6a
36       - c5a
37       - c5n
38       - c5
39     - key: karpenter.k8s.aws/instance-size
40       operator: In
41       values:
42       - xlarge
43       - 2xlarge
44       - 4xlarge
45     - key: kubernetes.io/os
46       operator: In
47       values:
48       - linux
49     ttlSecondsAfterEmpty: 600
50     ttlSecondsUntilExpired: 300
51     taints:
52     - effect: NoSchedule
53       key: karpenter.sh/legacy
54       value: "true"

```

2. Forced Deletion: For each Provisioner in your cluster:

- Delete the old Provisioner with: `kubectl delete provisioner <provisioner-name> --cascade=foreground`
- Wait as Karpenter deletes all the Provisioner's nodes. All nodes will drain simultaneously. New nodes are launched after the old ones have been drained.

3. Manual Rolling: For each Provisioner in your cluster:

- Add the following taint to the old Provisioner: `karpenter.sh/legacy=true:NoSchedule`
- For all the nodes owned by the Provisioner, delete one at a time as follows: `kubectl delete node <node-name>`

Update workload labels: Old alpha labels (`karpenter.sh/do-not-consolidate` and `karpenter.sh/do-not-evict`) [are deprecated](#), but will not be dropped until Karpenter v1. However, you can begin updating those labels at any time with `karpenter.sh/do-not-disrupt`.

Any pods that specified a `karpenter.sh/provisioner-name:DoesNotExist` requirement also need to add a `karpenter.sh/nodepool:DoesNotExist` requirement to ensure that the pods continue to not schedule to nodes unmanaged by Karpenter while migrating to v1beta1.

Ensure that there are no more Machine resources on your cluster. You should see `No resources found` when running the following command:

```
1 kubectl get machines
```

Once there are no more Machines on your cluster, it is safe to delete the other Karpenter configuration resources. You can do so by running the following commands:

```
1 kubectl delete provisioners --all
2 kubectl delete awsnodetemplates --all
```

As we are upgrading the existing karpenter, necessary tags to subnets and security should be in place , we can cross the tags to subnet and security groups.

Add tags to subnets and security groups

We need to add tags to our nodegroup subnets so Karpenter will know which subnets to use.

```
1 for NODEGROUP in $(aws eks list-nodegroups --cluster-name "${CLUSTER_NAME}" --query 'nodegroups' --output text);
2   aws ec2 create-tags \
3     --tags "Key=karpenter.sh/discovery,Value=${CLUSTER_NAME}" \
4     --resources "$(aws eks describe-nodegroup --cluster-name "${CLUSTER_NAME}" \
5       --nodegroup-name "${NODEGROUP}" --query 'nodegroup.subnets' --output text )"
6 done
```

Add tags to our security groups. This command only tags the security groups for the first nodegroup in the cluster. If you have multiple nodegroups or multiple security groups you will need to decide which one Karpenter should use.

```
1 NODEGROUP=$(aws eks list-nodegroups --cluster-name "${CLUSTER_NAME}" \
2   --query 'nodegroups[0]' --output text)
3
4 LAUNCH_TEMPLATE=$(aws eks describe-nodegroup --cluster-name "${CLUSTER_NAME}" \
5   --nodegroup-name "${NODEGROUP}" --query 'nodegroup.launchTemplate.{id:id,version:version}' \
6   --output text | tr -s "\t" ",")
7
8 # If your EKS setup is configured to use only Cluster security group, then please execute -
9
10 SECURITY_GROUPS=$(aws eks describe-cluster \
11   --name "${CLUSTER_NAME}" --query "cluster.resourcesVpcConfig.clusterSecurityGroupId" --output text)
12
13 # If your setup uses the security groups in the Launch template of a managed node group, then :
14
15 SECURITY_GROUPS="$(aws ec2 describe-launch-template-versions \
16   --launch-template-id "${LAUNCH_TEMPLATE%,*}" --versions "${LAUNCH_TEMPLATE#*,}" \
17   --query 'LaunchTemplateVersions[0].LaunchTemplateData.[NetworkInterfaces[0].Groups[0].SecurityGroupIds]' \
18   --output text)"
19
20 aws ec2 create-tags \
21   --tags "Key=karpenter.sh/discovery,Value=${CLUSTER_NAME}" \
22   --resources "${SECURITY_GROUPS}"
23
```

Update aws-auth ConfigMap

We need to allow nodes that are using the node IAM role we just created to join the cluster. To do that we have to modify the `aws-auth` ConfigMap in the cluster.

```
1 kubectl edit configmap aws-auth -n kube-system
```

You will need to add a section to the mapRoles that looks something like this. Replace the `${AWS_PARTITION}` variable with the account partition, `${AWS_ACCOUNT_ID}` variable with your account ID, and `${CLUSTER_NAME}` variable with the cluster name, but do not replace the `{{EC2PrivateDNSName}}`.

```
1 - groups:
2   - system:bootstrappers
3   - system:nodes
4   ## If you intend to run Windows workloads, the kube-proxy group should be specified.
5   # For more information, see https://github.com/aws/karpenter/issues/5099.
6   # - eks:kube-proxy-windows
7   rolearn: arn:${AWS_PARTITION}:iam::${AWS_ACCOUNT_ID}:role/KarpenterNodeRole-${CLUSTER_NAME}
8   username: system:node:{{EC2PrivateDNSName}}
```

The full aws-auth configmap should have two groups. One for your Karpenter node role and one for your existing node group.